

Real-time Object detection with YOLO

We talk with Joseph Redmon about his YOLO object detection algorithm which is making waves in the AI community.

Mithun Mohandas | mithun@digit.in

Q We saw your TED Talk in Vancouver, where you claimed that the #YOLO algorithm is lean. Is it lean enough to be implemented on, say, edge devices/IoT?

Joseph: Basically, there's a lot of re-search out there about making these neural networks smaller and faster by using a bunch of different optimisation techniques that you can use. So we've been doing a lot of work on that. If you remember the TED Talk, we'd demonstrated the algorithm on an iPhone and it was working pretty well.

Q But an iPhone is way powerful than most IoT devices.

Joseph: We're still working on stuff like that. Things like using network binarization and a bunch of optimisation techniques are used to make these neural networks fast. The good thing is that it's basically a straightforward neural network. There's a lot of research going on in a bunch of other domains, and similar techniques are used to make neural networks faster for classification and other problems. They all transfer over to YOLO. We've definitely been working to get on the cell phones, to make them faster and more usable. So that even folks without a big GPU can make use of it.

Q What's so different about the YOLO algorithm that makes it so lean?

Joseph: The approach to detection in YOLO is different from what people were doing in the past. In the past, you'd

extract a bunch of discrete patches from an image and run a classifier on each of those patches. It's sort of a sliding-window based approach, or more recently, people have been using selective search or RCNN or faster RCNN. These all use a technique where you are sort of running a classifier on a bunch of places in an image. And to do that you are running a model hundreds of thousands of times on the same image. That's where the "You Only Look Once" aspect comes from. We're just going to run a single neural network on the image and have it predict the whole detection pipeline. It's sort of a reimagining of the problem.



Joseph Redmon,
Computer Scientist

Q So YOLO can achieve the same level of accuracy with a smaller training set?

Joseph: I would say that right now, training data is one of the big limiting factors. We always want more training data. More training data always makes it better. That being said, this is also one of the things that we're trying to think about. In practice, what people really want to do is take a couple of pictures of something and have their smartphone or camera recognise that specific object. This is a problem of single-shot learning or few-shot learning that businesses and even consumers are interested in. And this is something I've been thinking of, about how we can take this domain and wrap it into the YOLO system. When it comes down to it, people don't want to sit and label tens of thousands of images they just want it to work. We don't have a way of making that work just yet, but we have some ideas on how to approach this.

Q How about the approach that AlphaGo Zero used where it was just fed the rules and then it learnt by playing against itself? Can that approach reduce the reliance on massive data sets?

Joseph: AlphaGo Zero is super interesting but there isn't the same kind of ruleset in computer vision since we're mostly dealing with pictures in the real world. One possible thing that comes close is taking a computer game engine and using that to generate training data. It's pretty easy to figure out where objects are when using game engines and you can generate an unlimited amount of data using this approach. One of the limiting factors is getting a really good game engine, then implementing it to produce really high-quality frames for the training set. Engines aren't able to generate photorealistic images just yet, but it definitely has some potential. Interestingly, some Generative Adversarial Networks (GANs) could in theory be able to generate photorealistic images, but it remains to be seen.

Q Would they be accurate? Real world images have artifacts, whereas with a game engine, you get crisp images.

Joseph: Exactly. The real world has all these little factors that you can't really simulate really well. In one sense, it can be really good for pre-training. So if you don't have many images, then you can generate some images for pre-training. And then take your neural network and fine tune it with your real-world data set. This is a standard that some people are already using in sort of a different way. Right now, if you want to build a neural net, then you train it first on ImageNet. Once you have the weights, you take your model and then train it on whatever you want to focus on. It ends up being a pretty good starting place for your neural network since it builds a good extractor. So you don't have to start from scratch. **Q**